

fixtures/handoff/ – T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/orchestration/handoff.py` (contract `specs/004-fast-dev-cycles/contracts/agent-registry-and-handoff.md`, clauses HO-001/HO-002; data-model.md §9.2 "Handoff Record"; plan.md T-B04; research.md DEC-34). Task T125 (`tasks.md`) writes ONLY this RED test + these fixtures, scoped to `write / validate` (HO-001's write obligation and HO-002's partial-artefact re-hash-and-flag obligation); the tool itself is T133, a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `handoff.py`). `resume-check / HO-003` (external-dependency revalidation) is deliberately OUT OF SCOPE here — that is T126/T-B05's own RED test (`test_resume_revalidate_red.sh`), never duplicated in this file.

Layout

Four self-contained scenario directories, one per the task's own named check (T125's task line, quoted verbatim from `tasks.md`): "a killed fixture agent leaves no handoff; golden: record round-trips; golden-bad: artefact hash not matching disk is flagged; negative control — the CT-5 addition: a partial artefact legitimately re-written with its recorded hash updated validates PASS".

Directory	Kind	What it proves
ho_killed_no_handoff/	red-baseline	today, a killed fixture agent leaves NO <code>handoff.json</code> at all — <code>orchestration/handoff.py</code> does not exist, so HO-001's "on crashed detection a final [handoff] one is written" obligation is wholly unmet
ho_golden_roundtrip/	golden-good	<code>handoff.json</code> is self-consistent (<code>handoff_id</code> / <code>body_hash</code> recompute correctly from its own body) AND both <code>partial_artefacts</code> entries' on-disk bytes hash-match their recorded <code>content_address</code> — the record round-trips
ho_golden_bad_tampered_artefact/	golden-bad	<code>notes.md</code> 's CURRENT bytes were altered AFTER <code>handoff.json</code> was written, without the record being updated — its <code>content_address</code> no longer matches; HO-002: "verify re-hashes [partial artefacts] and exits 1 on any difference"

Directory	Kind	What it proves
<code>ho_negctrl_legit_rewrite/</code>	negative-control (CT-5 addition)	<code>notes.md</code> 's CURRENT bytes genuinely differ from an earlier snapshot (<code>notes_as_first_written.md</code>) — a real rewrite occurred — but <code>handoff.json</code> 's recorded <code>content_address</code> was correspondingly UPDATED at the SAME phase-boundary write to match the NEW bytes; <code>validate</code> must PASS, distinguishing a legitimate re-record from genuine tampering (the golden-bad fixture above)

Every sha256 embedded in every `handoff.json` is the REAL digest of the file it names, computed at fixture-authoring time by `hashlib.sha256` and independently RE-VERIFIED, live, by the already-landed `constitution/scripts/fastcycle/lib/fc_common.py` `body-hash --verify` (a control needle: the established tool, not this fixture author's own reimplementaion, confirms every `handoff.json` 's stored `body_hash` matches its canonical body — `bash test_handoff_red.sh` reruns this check on every invocation, so a future hand-edit that desyncs a `handoff.json` from its own `body_hash` is caught rather than silently poisoning the fixture).

1 3 3

`handoff.json` wire format (UNCONFIRMED, binding-if-adopted on T)

`data-model.md` §9.2 names the Handoff Record entity's FIELDS (`handoff_id`, `agent_key`, `item_id`, `alias`, `model`, `effort`, `phase`, `verified`, `pending`, `partial_artefacts`, `external_deps`, `effects_performed`, `written_at`) but does not pin the exact JSON shape those fields are serialized into. This fixture set — like the established precedent in `fixtures/evidence_ref/` , `fixtures/verdict_cache/` , and `test_governance_subset_red.sh` 's own

"UNCONFIRMED... DEFINED here" section — DEFINES one concrete, defensible shape and flags it explicitly as binding-if-adopted, not as a claim the contract or data-model already specifies it:

- **verified is a list of bare EvidenceReference id strings** (data-model.md §9.2 literally: "list of EvidenceReference ids"), e.g. ["FC-H0-VERIFIED-001"] . plan.md's own T-B04 entry records "Depends on: T-E05 for reference format (can start with inline hashes until it lands)" — T-E05/T117 (context/evidence_ref.py) has not landed either (confirmed directly: constitution/scripts/fastcycle/context/ holds only anchor_citations.py , a separate T039 task), so these fixture ids are placeholder labels only, never resolved against a real evidence-reference store by anything this test checks.
- **pending is a list of {step, precondition} objects**, per data-model's own field description ("ordered list of {step, precondition} ") — no further structure invented.
- **partial_artefacts is a list of {path, content_address} objects** where path is repo-relative-to-the-scenario-directory and content_address is the §0 ContentAddress type ("sha256:<64 hex>" , prefixed) of that path's bytes — data-model's own field description verbatim ("list of {path, ContentAddress} ").
- **external_deps and effects_performed** carry the closed-vocabulary kind fields data-model.md §9.2 names, but no fixture here exercises resume-check 's re-hashing of them (T126/T-B05's job) — every external_deps entry in these fixtures is marked with the literal reason string "NOT-EXERCISED-BY-T125-VALIDATE-SEE-T126" in place of a real fingerprint, so a future reader (human or agent) never mistakes an unexercised placeholder for a checked value.
- **Content addresses are "sha256:<64 hex>" -prefixed** (data-model.md §0's own ContentAddress format).
- **handoff_id and body_hash are TWO DEPENDENT, NON-CIRCULAR hashes, computed in two passes, reusing the SAME already-landed fc_common.py canonicalization convention (canon()) :**

`json.dumps(sort_keys=True, separators=(",", ":"), ensure_ascii=False)`) for BOTH rather than inventing a second one (§11.4.227):

1. **Pass 1** — `handoff_id` (data-model's "ContentAddress" of the record body"): `"sha256:" + sha256(canon(doc excluding handoff_id, body_hash, run_meta))` , computed with `handoff_id` and `body_hash` still ABSENT from the doc. This is the record's semantic-identity hash — content changes to any field (`phase` , `partial_artefacts` , `pending` , ...) change `handoff_id` .
 2. **Pass 2** — `body_hash` : `handoff_id` is inserted into the doc, THEN the established `fc_common.py` C-002 convention's own `body_hash_of()` is applied — `sha256(canon(doc excluding ONLY body_hash, run_meta))` — now hashing a doc that DOES include `handoff_id` . This is EXACTLY what `fc_common.py` `body-hash --verify` recomputes, so it is a genuine, reusable control needle, never a bespoke reimplement. The two hashes therefore differ by design (pass 2's input is pass 1's input plus one field, `handoff_id` , so a naive "they must be equal" expectation would be wrong) — `handoff_id` is the record's own semantic identity; `body_hash` is the whole-envelope integrity check over the bytes as actually stored on disk, reusing the identical hashing primitive `fc_common.py` already establishes rather than a second, divergent one.
- `time_source` on `written_at` uses the closed §0 `Instant` vocabulary; every fixture here uses `"event_occurred"` (a phase-boundary write is, by construction, the moment the event it describes occurred — never `db_write` / `file_mtime` /etc., which describe a DIFFERENT kind of write).

What this RED test does NOT cover

`resume-check` / HO-003 (re-hashing `external_deps` , invalidating dependent `verified` facts, the five "Safe to Resume?" failure modes) is T126's own RED test against T-B05, never duplicated or pre-empted here — see `test_resume_revalidate_red.sh` . `write` 's exact CLI invocation shape (`handoff.py write --handoff <path> --out <result.json>` , per the contract's Components section) is likewise not exercised against a real binary in this file (T133 has not landed); this file's independent oracle only proves what a

conformant write / validate implementation MUST produce, over these checked-in fixtures, so T133 has a concrete, self-validated target to turn GREEN against.